

MPOO 2027-1 · GRUPO 6 · CLASE 2 · LUNES 24 DE AGOSTO

El paradigma orientado a objetos

Hoy dejamos de escribir **pasos** y empezamos a construir **cosas**: qué es un objeto, qué es una clase, y la anatomía completa de una clase Java.

1 Funciones vs objetos

2 Objeto: estado + comportamiento

3 Clase: el molde

4 Atributos y métodos

5 Nombres, palabras reservadas y comentarios

6 Anatomía + demo

Lo que ya saben: estructurado

```
double velocidad = 0;
double gasolina = 40;

double acelerar(double vel, double gas, double kmh) {
    if (gas > 0) { return vel + kmh; }
    return vel;
}

// tú le acarreas los datos y tú guardas el resultado:
velocidad = acelerar(velocidad, gasolina, 20);
```

- La función **no sabe de quién** es esa velocidad: se la pasas tú, cada vez.
- Si olvidas el `velocidad =` de la izquierda, **no pasa nada y nadie avisa**.
- `velocidad` y `gasolina` son del mismo coche **solo en tu cabeza**: en el código, nada las une.

A dónde vamos: orientado a objetos

```
public class Coche {
    double velocidad;
    double gasolina;

    void acelerar(double kmh) {
        if (gasolina > 0) { velocidad = velocidad + kmh; }
    }
}

// no le pasas sus datos: ya son suyos
miCoche.acelerar(20);
```

- El método solo recibe **lo nuevo** (cuánto acelerar); la velocidad y la gasolina **ya viven ahí**.
- Nada que olvidar guardar: el coche **actualiza su propio estado**.
- "Esta velocidad es de este coche" ya no vive en tu cabeza: **lo garantiza el lenguaje**.

La prueba de fuego: con dos coches, `acelerar(velocidad1, gasolina2, 20)` **compila sin quejarse**, mezclaste dos coches y nadie te avisó. En objetos, esa confusión **ni siquiera se puede escribir**: `coche1.acelerar(20)` solo toca lo de `coche1`.

Como lo venían haciendo

```
double saldoAna  = 500;
double saldoLuis = 300;

// retirar 200 de la cuenta de Ana
if (saldoAna >= 200) {
    saldoAna = saldoAna - 200;
}
```

- Cliente nuevo = **variable nueva**: saldoPedro, saldoMaría...
- La regla "no retirar de más" se **repite en cada retiro** del programa.
- Un if olvidado en un solo lugar = **dinero fantasma**.

Transformado a objetos

```
public class CuentaBancaria {
    String titular;
    double saldo;

    void retirar(double monto) {
        if (saldo >= monto) {
            saldo = saldo - monto;
        }
    }
}
```

```
ana.retirar(200); // la regla se aplica sola
```

- Cliente nuevo = **new**, cero variables nuevas.
- La regla vive **una sola vez**, dentro de retirar().
- **Nadie puede saltársela**: no hay retiro que no pase por ahí.

La regla viaja **con los datos**. Eso es lo que compra el paradigma.

Un objeto **sabe cosas y sabe hacer cosas**

Un coche

Sabe: su color, su velocidad actual, cuánta gasolina le queda.

Sabe hacer: acelerar, frenar, encender.

Un círculo

Sabe: su radio.

Sabe hacer: calcular su área, calcular su perímetro.

Una cuenta bancaria

Sabe: su saldo, su titular.

Sabe hacer: depositar, retirar (y negarse si no alcanza).

Lo que sabe = estado

Los datos que el objeto carga consigo. Cambian con el tiempo: la velocidad sube, el saldo baja.

Lo que sabe hacer = comportamiento

Las operaciones que ofrece. Trabajan **sobre su propio estado**: acelerar cambia *su* velocidad, no la de otro coche.

La clase es el **plano**

Se escribe **una vez**. Dice qué estado y qué comportamiento va a tener todo coche, sin ser ningún coche en particular.

```
public class Coche {  
    String color;  
    double velocidad;  
    void acelerar(double kmh) {...}  
}
```

Una clase define un **tipo de dato nuevo**: igual que `double` es un tipo y 3.14 es un valor, `Coche` es un tipo y cada coche concreto es un valor de ese tipo.

Los objetos son las **instancias**

Se fabrican con **new**, las que quieras. Cada una con **su propio estado**:

miCoche

color = "rojo" · velocidad = 120.0

tuCoche

color = "azul" · velocidad = 0.0

cocheDeRenta

color = "blanco" · velocidad = 80.0

Acelerar **miCoche no mueve **tuCoche**:**
cada instancia vive su vida.

```
public class Coche {  
  
    // ATRIBUTOS: el estado  
    String color;  
    double velocidad;  
  
    // MÉTODO: el comportamiento  
    void acelerar(double kmh) {  
        velocidad = velocidad + kmh;  
    }  
}
```

Y desde afuera, todo se usa con el **punto**:

```
miCoche.acelerar(20);
```

- **Atributo**: variable que vive **dentro del objeto**. Cada instancia tiene su copia.
- **Método**: función que vive **dentro de la clase** y puede leer y cambiar los atributos directamente (mira: usa velocidad sin pedirla como parámetro).
- **La firma** void acelerar(double kmh) es el **contrato**: qué recibe, qué devuelve, cómo se llama. Es todo lo que quien lo usa necesita saber.

Compárenlo con el estructurado

Ya no es acelerar(velocidad, gasolina) pasando todo a mano: el método **ya vive donde viven los datos**. Eso es juntar estado y comportamiento.

```
// suelto, como antes
double radio1 = 2.0;
double radio2 = 5.0;

double area(double radio) {
    return 3.1416 * radio * radio;
}
// area(radio1), area(radio2), ...
```

```
public class Circulo {
    double radio;

    double area() {
        return Math.PI * radio * radio;
    }
}
// c1.area() ← sin parámetro: ya sabe su radio
```

1 Variables sueltas → atributos

radio1, radio2 desaparecen: cada círculo **carga su propio radio**.

2 Funciones → métodos sin parámetros

area(radio) se vuelve area(): ya no le pasas los datos, **vive donde viven los datos**.

3 Cada juego de valores → una instancia

Lo que era radio1 y radio2 ahora es c1 y c2, creados con new.

Guárdense la receta: aplicarla a un objeto del mundo real **es** la P2 del 2 de septiembre. Y este Circulo lo construimos en vivo en un momento.

Abstracción • quedarte con lo esencial

Un coche real tiene miles de piezas. Tu clase Coche modela **solo lo que tu programa necesita**: para un simulador de tráfico, velocidad y posición; ni tornillos, ni asientos, ni el color del tablero.

Abstraer es decidir qué ignorar. Es la habilidad número uno de esta materia, y la practican en la P2, el miércoles 2 de septiembre.

Encapsulamiento • esconder el interior

Manejas con **volante y pedales**: nunca tocas la inyección de gasolina directo, y por eso no la puedes descomponer. La clase igual: expone operaciones seguras (`acelerar()`, `retirar()`) y esconde sus datos.

La cuenta bancaria **se niega** a retirar si no alcanza el saldo: la regla vive **dentro**, nadie la puede saltar.

Volante y pedales a la vista, motor escondido.

Identificadores: las reglas

- Empieza con **letra**, `_` o `$`; después también dígitos.
- **Sensible a mayúsculas**: `velocidad`, `Velocidad` y `VELOCIDAD` son tres nombres distintos.
- No puede ser una **palabra reservada**.
- Acentos y ñ son **legales** (Java es Unicode)... y una **pésima idea**: algún día ese archivo cruza una frontera.

NOMBRE	¿VÁLIDO?
miCoche	Sí
2doIntento	No : empieza con dígito
mi-coche	No : el guion es una resta
class	No : reservada
años	Sí , pero no lo hagas

~50 palabras reservadas

Son del lenguaje y no se pueden usar como nombres. Las de hoy:

classpublicstaticvoidnewreturnifelsewhileforint

doublebooleanfinal

Curiosidades: `true`, `false` y `null` técnicamente son literales, no palabras clave; y `const` y `goto` están reservadas **sin usarse nunca**: Java se las apartó de C "por si acaso", hace 30 años, y ahí siguen.

La convención que se califica
Clases en PascalCase: `Coche`, `CuentaBancaria`.
Variables y métodos en camelCase: `miCoche`, `calcularArea`.
Constantes: `VELOCIDAD_MAXIMA`.

// de línea

```
// hasta el fin de línea  
double radio = 2.5; // metros
```

/* */ de bloque

```
/* varias líneas.  
   Ojo: no se pueden  
   anidar. */
```

/** */ de documentación

```
/** Un coche del simulador.  
    De aquí salen las páginas  
    de documentación de Java. */
```

El buen comentario dice **por qué**

```
// suma 20 no aporta nada: eso ya lo dice el código. // el sensor entrega km/h,  
no m/s salva al que sigue.
```

El espacio en blanco no significa nada

La indentación es **para humanos**: Java no es Python. El compilador solo ve los ; y las llaves. Aun así, indentamos siempre, porque el código se lee mil veces más de lo que se escribe.

```
public class Coche {  
  
    String color;  
    double velocidad;  
  
    void acelerar(double kmh) {  
        velocidad = velocidad + kmh;  
    }  
  
    public static void main(String[] args) {  
        Coche miCoche = new Coche();  
        miCoche.color = "rojo";  
        miCoche.acelerar(20);  
        System.out.println(miCoche.color  
            + " a " + miCoche.velocidad);  
    }  
}
```

- **public class Coche** obliga a que el archivo se llame **Coche.java**. El nombre de la clase **es** su dirección en disco.
- **Atributos primero, métodos después**: convención, no regla; pero así lo lee todo el mundo.
- **main** es la **puerta de entrada**: la ejecución empieza ahí. Es el mismo main de su HolaMundo.
- **new Coche()** fabrica la instancia y ejecuta el **constructor** (hoy, el que Java regala; los nuestros, próxima unidad).
- **El punto** entra al objeto: `miCoche.color`, `miCoche.acelerar(20)`.
- Imprime: **rojo a 20.0**

Vamos a construir `Circulo`

- Clase `Circulo` con atributo `double radio`.
- Método `double area()` con `Math.PI * radio * radio`.
- Método `double perimetro()`.
- En `main`: **dos** círculos con radios distintos, imprimimos sus áreas.

Predicciones al chat, antes de correr

- Si cambio el radio de `c1`... ¿cambia el área de `c2`?
- ¿Qué imprime `c1.area()` si nadie le puso radio?
- ¿Cuántos objetos hay si escribo `new Circulo()` tres veces?
- ¿Y si nombro el archivo `circulo.java` con minúscula?

Primero apuestan, luego corremos.

Equivocarse en la predicción **es el aprendizaje.**

1 El archivo no se llama como la clase

```
public class Coche { ... } // guardado como coche.java
```

JAVA DICE
class Coche is public, should be declared in a file named Coche.java

Arreglo: renombra el archivo a Coche.java, mayúscula incluida.

2 Una mayúscula cambiada

```
system.out.println("hola");
```

JAVA DICE
cannot find symbol: variable system

Arreglo: es System. Java nunca perdona una mayúscula.

3 Falta un punto y coma

```
double radio = 2.0      ← aquí faltó  
double area;           ← aquí truena
```

JAVA DICE
';' expected

Arreglo: el ; perdido casi siempre está en la **línea anterior** a la que marca el error.

4 Código suelto fuera de un método

```
public class Coche {  
    velocidad = 20;    ← instrucción huérfana  
}
```

JAVA DICE
illegal start of type

Arreglo: las instrucciones viven **dentro de métodos**; en el cuerpo de la clase solo se declaran atributos y métodos.

Regla de la casa: el mensaje de error **se lee completo** antes de preguntar.
La mitad de las veces trae la respuesta.

1 · ¿Clase u objeto?

- **Coche** → ¿?
- **El Tsuru rojo de mi tío** → ¿?
- **CuentaBancaria** → ¿?
- **La cuenta 0042 de Ana con \$500** → ¿?
- **String** → ¿?
- **"hola"** → ¿?

2 · Modelen un FocoInteligente

Al chat, en una línea, formato libre:

- **2 atributos** (¿qué sabe un foco?)
- **2 métodos** (¿qué sabe hacer?)

Leo respuestas en voz alta y las convertimos en clase Java al vuelo. No hay respuesta única: **abstraer es elegir**, y cada elección se defiende.

Hoy se llevan

Objeto = **estado + comportamiento** · clase = **el molde**, objetos = **instancias** con new · atributos y métodos · las reglas de nombres y comentarios · la anatomía completa de una clase Java.

Miércoles 26 · laboratorio presencial

Git, GitHub y **su primera entrega con pull request**: la P1, con el HolaMundo personalizado del Lab 0. Traigan su laptop con el entorno de las Partes 0 a 2 funcionando.

AHORITA, al chat: usuario: su-usuario-de-github

Hoy mismo les llega la invitación de colaborador. Acéptenla antes del miércoles: es el requisito para entregar.